Module: Flow Control

Phase: Core Python
Previous module: Variables Datatypes Operators IO
Next module: Strings

Module Outcome:

- Decide like an ATM: withdraw only if the PIN and balance allow it
- Repeat work with `while` / `for` instead of copy-pasting the same lines
- Build a small menu (pass/fail, cart check, retry) that does not run in a straight line

Why flow control?

Programs rarely run in one straight line. A **student portal** must check login, an **ATM** repeats until user exits, and a **task manager** loops through tasks.

🏦 Bank 📱 UPI 👤 Employee 📦 Order 💰 Payroll

Where used?
- **HDFC Bank** ATM — PIN retries, withdraw only if balance allows
- College apps — pass/fail grades, marks menus
- **Swiggy** — cart checkout loops, stock checks
- **Employee Management Portal** — overtime flags, salary slab branches
- **Apollo Hospitals** — ticket priority menus, appointment status

Flow control lets you:

- **Decide** — run code only when a condition is true (`if`)
- **Repeat** — run code many times (`for` , `while`)
- **Jump** — leave early (`break`) or skip one round (`continue`)

if / elif / else

```
marks = int(input("Enter marks: "))

if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 35:
    grade = "C"
else:
    grade = "Fail"

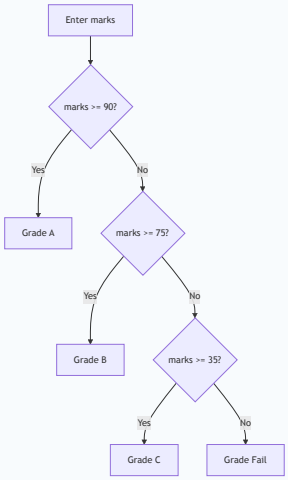
print(f"Grade: {grade}")
```

Rules:

- Colon `:` after condition

- Indented block (usually 4 spaces) — **indentation is syntax in Python**
- `elif` = else if; `else` catches everything not matched above

Visual — how Python chooses a branch (student grades):



Key takeaways:
- Checks run **top → bottom**; first `True` branch wins
- Later `elif`s are skipped once one branch runs
- `else` is the safety net when nothing above matched
- Same idea as ATM: check invalid amount → insufficient → then withdraw

Only **one** box on the right runs — the first `True` condition wins; `else` is the last safety net.

for loop

Use when you know **how many times** or want to visit **each item**.

```
for i in range(1, 6):      # 1, 2, 3, 4, 5
    print(f"Task {i}: pending")
```

while loop

Use when repetition depends on a **condition** staying true.

```
pin = ""
while pin != "1234":
    pin = input("Enter PIN: ")
print("Access granted")
```

range()

```
range(5)        # 0, 1, 2, 3, 4
range(2, 8)     # 2, 3, 4, 5, 6, 7  — stop excluded
range(0, 10, 2) # 0, 2, 4, 6, 8
```

break and continue

```
for n in range(1, 11):
    if n == 5:
        continue    # skip printing 5
    if n == 8:
        break       # stop loop entirely
    print(n)
# prints 1 2 3 4 6 7
```

Menu pattern (ATM / portals)

Most CLI apps in training use:

```
while True:
    show menu
    read choice
    if choice == 1: do action
    elif choice == 2: do action
    elif choice == 0: break
    else: invalid
```

This is the skeleton for ATM, expense tracker, and task manager projects.

Important Concepts and Definitions

Term	Definition
Condition	Boolean expression controlling branch or loop
Branch	One path of if/elif/else
Iteration	One execution of loop body
Infinite loop	Loop condition never becomes false
Sentinel value	Special input meaning "stop" (e.g. 0 to exit)
Nested loop	Loop inside another loop
Control flow	Order in which statements execute
Block	Indented group of statements
Loop variable	Variable receiving each value in for-loop

Flowchart symbols (text):

```
Decision:  <condition?> ----yes----> [action]
              |
             no
              |
            [other action]
```

Step-by-Step Notes (With Annotated Code)

Step 1: Pass/fail — student management

```
# pass_fail.py

marks = float(input("Enter final marks (out of 100): "))

if marks < 0 or marks > 100:
    print("Invalid marks entered.")
elif marks >= 35:
```

```
    print(f"Pass — scored {marks}")
else:
    print(f"Fail — scored {marks}. Minimum is 35.")
```

Note: `or` combines invalid range checks.

Note:
- Wrong: Missing colon (`if x > 5`), inconsistent indentation, or putting `elif` after `else`.
- Prefer: `if x > 5:` with 4 spaces; order is if → elif → else (else last).
- Wrong: Leaving an empty block under `if` / `else`.
- Prefer: Use `pass` as a placeholder when needed.

Check marks 72 for invalid range, pass, or fail using if/elif/else.

```
marks = 72
if marks < 0 or marks > 100:
    print("Invalid marks")
elif marks >= 35:
    print("Pass")
else:
    print("Fail")
```

Expected Output:

```
Pass
```

Classify age 62 as Minor, Adult, or Senior with elif branches.

```
age = 62
if age < 18:
    print("Minor")
elif age < 60:
    print("Adult")
else:
    print("Senior")
```

Expected Output:

```
Senior
```

Step 2: Employee login (simplified)

```
# login_demo.py — hard-coded credentials for lab only

VALID_USER = "admin"
VALID_PASS = "secret123"

username = input("Username: ")
password = input("Password: ")

if username == VALID_USER and password == VALID_PASS:
    print("Welcome to Employee Portal")
```

```
else:
    print("Invalid credentials. Contact HR.")
```

Note:
- Wrong: Using `=` when you meant to compare — logic always wrong or SyntaxError.
- Prefer: `==` to compare, `=` to assign.

Compare entered credentials to stored username/password and print Welcome or Invalid.

```
username = "priya"
password = "ksr123"
entered_user = "priya"
entered_pass = "ksr123"
if entered_user == username and entered_pass == password:
    print("Welcome")
else:
    print("Invalid")
```

Expected Output:

```
Welcome
```

Run separate username and PIN checks and print which check failed.

```
username = "priya"
pin = "4321"
entered_user = "priya"
entered_pin = "0000"
if entered_user != username:
    print("Username failed")
elif entered_pin != pin:
    print("PIN failed")
else:
    print("Welcome")
```

Expected Output:

```
PIN failed
```

Common Error: Using `=` inside `if` (`if x = 5`) — that is assignment, not comparison. Use `==`.

Step 3: for loop — multiplication table

```
n = int(input("Table for which number? "))
print(f"--- Multiplication table of {n} ---")

for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

Note:
- Wrong: Off-by-one with `range` (loop one short or one long).
- Prefer: Remember the stop value is exclusive (`range(1, 11)` → 1…10).

Print the multiplication table of 7 for 1 through 10.

```
for i in range(1, 11):
    print(f"7 x {i} = {7 * i}")
```

Print even numbers from 2 to 20 on one line using range with step 2.

```
for n in range(2, 21, 2):
    print(n, end=" ")
print()
```

Expected Output:

```
2 4 6 8 10 12 14 16 18 20
```

Remember: `range` stop is **exclusive** — `range(1, 11)` gives 1…10, not 11.

Step 4: while loop — PIN retries (max 3)

```
CORRECT_PIN = "4321"
attempts = 0
max_attempts = 3

while attempts < max_attempts:
    pin = input("Enter ATM PIN: ")
    if pin == CORRECT_PIN:
        print("PIN accepted.")
        break
    else:
        attempts += 1
        remaining = max_attempts - attempts
        print(f"Wrong PIN. {remaining} attempt(s) left.")
else:
    # while-else: runs if loop ended without break
    print("Card blocked. Visit branch.")
```

Note:
- Wrong: Infinite `while` — the condition never moves toward exit.
- Prefer: Ensure a variable in the condition changes (counter, flag, or `break`).

Simulate up to three PIN attempts with while-else; break on correct PIN or block the card.

```
CORRECT_PIN = "2580"
attempts = ["1111", "2222", "2580"]  # demo tries
i = 0
while i < 3:
    pin = attempts[i]
    i += 1
    if pin == CORRECT_PIN:
```

```
        print("Access granted")
        break
else:
    print("Card blocked")
```

Expected Output:

```
Access granted
```

Loop demo amounts until you find a positive value and print Accepted.

```
amounts = [-50, 0, 250]  # demo inputs until positive
for amount in amounts:
    if amount > 0:
        print(f"Accepted: ₹{amount}")
        break
```

Step 5: continue — skip invalid expenses

```
total = 0
for _ in range(3):   # _ means "I don't need loop variable name"
    amt_str = input("Expense amount: ")
    if not amt_str.replace(".", "", 1).isdigit():
        print("Skipped invalid entry.")
        continue
    total += float(amt_str)

print(f"Valid expense total: ₹{total:.2f}")
```

Sum only numeric expense strings from a list, skipping invalid entries with continue.

```
raw = ["120", "abc", "85.5", "xx", "40"]
total = 0.0
for item in raw:
    # String check now; try/except comes in Exceptions week
    if not item.replace(".", "", 1).isdigit():
        continue
    total += float(item)
print(f"Sum of valid amounts: ₹{total}")
```

Print numbers 1–15 skipping multiples of 3 using continue.

```
for n in range(1, 16):
    if n % 3 == 0:
        continue
    print(n, end=" ")
print()
```

Step 6: Nested if — task priority

```
priority = int(input("Priority (1=high, 3=low): "))
hours = float(input("Estimated hours: "))
```

```
if priority == 1:
    if hours > 8:
        print("Split high-priority task across days.")
    else:
        print("Schedule today.")
elif priority == 2:
    print("Schedule this week.")
else:
    print("Backlog.")
```

Use nested if to print Distinction, Pass, or Fail for marks 82.

```
marks = 82
if marks >= 35:
    if marks >= 75:
        print("Distinction")
    else:
        print("Pass")
else:
    print("Fail")
```

Expected Output:

```
Distinction
```

Check a positive withdrawal amount against balance with nested if and print the outcome.

```
balance = 5000
amount = 1200
if amount > 0:
    if amount <= balance:
        print("Withdrawal allowed")
    else:
        print("Insufficient funds")
else:
    print("Invalid amount")
```

Step 7: ATM menu skeleton (full pattern)

```
# atm_menu.py — simplified balance demo

balance = 5000.0

while True:
    print("\n===== ATM MENU =====")
    print("1. Check Balance")
    print("2. Deposit")
    print("3. Withdraw")
    print("0. Exit")
    choice = input("Choose option: ")

    if choice == "1":
        print(f"Balance: ₹{balance:.2f}")

    elif choice == "2":
        amount = float(input("Deposit amount: "))
        if amount > 0:
```

```
            balance += amount
            print("Deposit successful.")
        else:
            print("Amount must be positive.")

    elif choice == "3":
        amount = float(input("Withdraw amount: "))
        if amount <= 0:
            print("Invalid amount.")
        elif amount > balance:
            print("Insufficient balance.")
        else:
            balance -= amount
            print(f"Withdrawn ₹{amount:.2f}. New balance: ₹{balance:.2f}")

    elif choice == "0":
        print("Thank you. Remove card.")
        break

    else:
        print("Invalid choice. Try again.")
```

Note:
- Wrong: Comparing menu `input()` to an int (`choice == 1`) when choice is still a string.
- Prefer: Compare strings (`choice == "1"`) or convert deliberately with `int(choice)`.

Demo deposit, withdraw, overdraft check, and print final ATM balance.

```
balance = 5000
balance += 1000
print(f"After deposit: ₹{balance}")
withdraw = 2000
if withdraw <= balance:
    balance -= withdraw
    print(f"After withdraw: ₹{balance}")
overdraft = 10000
if overdraft > balance:
    print("Overdraft blocked")
print(f"Final balance: ₹{balance}")
```

Track balance updates and record the last deposit or withdraw action.

```
balance = 5000
last_action = "None"
balance += 500
last_action = "Deposited ₹500"
balance -= 200
last_action = "Withdrew ₹200"
print(f"Balance: ₹{balance}")
print(f"Last action: {last_action}")
```

Step 8: Task manager menu (add + list)

```
# task_manager_v1.py

tasks = []   # list comes in detail later — accept for now
```

```
while True:
    print("\n--- Task Manager ---")
    print("1. Add task")
    print("2. List tasks")
    print("0. Exit")
    choice = input("Option: ")

    if choice == "1":
        title = input("Task title: ")
        if title.strip():          # strip() preview — strings module
            tasks.append(title)
            print("Task added.")
        else:
            print("Title cannot be empty.")

    elif choice == "2":
        if not tasks:
            print("No tasks yet.")
        else:
            print("Your tasks:")
            for index, task in enumerate(tasks, start=1):
                print(f"  {index}. {task}")

    elif choice == "0":
        break

    else:
        print("Invalid option.")
```

Pop the last task from a list if any exist and print what was deleted and what remains.

```
tasks = ["Login page", "API", "Deploy"]
if tasks:
    cancelled = tasks.pop()
    print(f"Deleted: {cancelled}")
print("Remaining:", tasks)
```

Simulate a mini student portal that adds names, lists them, and handles blank input.

```
names = []
actions = [("1", "Priya"), ("1", ""), ("1", "Ravi"), ("2", None), ("0", None)]
for choice, value in actions:
    if choice == "1":
        if value and value.strip():
            names.append(value.strip())
            print(f"Added: {value.strip()}")
        else:
            print("Blank name rejected")
    elif choice == "2":
        print("Names:", names)
    elif choice == "0":
        print("Exit")
        break
```

Practice Exercises

Basic

1. Print whether a number is even or odd (`n % 2 == 0`).

Model answer:

```
n = int(input("Number: "))
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

2. Print numbers 1 to 20 using `for` and `range`.

Model answer:

```
for i in range(1, 21):
    print(i)
```

3. Sum numbers 1 to 100 using a loop.

Model answer:

```
total = 0
for i in range(1, 101):
    total += i
print(total)  # 5050
```

4. Ask age; print "Minor" if `< 18` else "Adult".

Model answer:

```
age = int(input("Age: "))
if age < 18:
    print("Minor")
else:
    print("Adult")
```

Intermediate

1. Grade calculator: marks → A/B/C/Fail with `elif` chain.

Model answer:

```
marks = int(input("Marks (0–100): "))
if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
elif marks >= 50:
    print("Grade: C")
else:
    print("Fail")
```

2. Number guessing: hard-code secret; 5 attempts with while; hints higher/lower.

Model answer:

```
secret = 7
attempts = 0
while attempts < 5:
    guess = int(input("Guess (1–10): "))
    attempts += 1
    if guess == secret:
        print("Correct!")
        break
    elif guess < secret:
        print("Higher")
    else:
        print("Lower")
else:
    print("Out of attempts. Secret was", secret)
```

3. Expense menu: add expense, show total, exit — use while True.

Model answer:

```
total = 0.0
while True:
    print("1 Add expense  2 Show total  3 Exit")
    choice = input("Choice: ")
    if choice == "1":
        amount = float(input("Amount ₹: "))
        total += amount
        print("Added.")
    elif choice == "2":
        print(f"Total: ₹{total:.2f}")
    elif choice == "3":
        print("Bye")
        break
    else:
        print("Invalid choice")
```

4. Print multiplication table only for even multipliers 2–10.

Model answer:

```
n = int(input("Table for: "))
for m in range(2, 11, 2):
    print(f"{n} x {m} = {n * m}")
```

Interview-Oriented

1. When use `for` vs `while`?

Model answer: Use **`for`** when you know the sequence or count upfront (`range`, list items, string chars). Use **`while`** when you loop until a condition changes — menus, retries, “keep going until user exits,” reading until sentinel. Prefer `for` when either works cleanly; use `while` for open-ended control.

2. What does `break` do inside nested loops? (Only innermost loop unless labeled — Python has no goto.)

Model answer: `break` exits only the **innermost** loop that contains it. Outer loops keep running. Python has no labeled break like some languages; to exit multiple levels, use a flag, `return` (once you know functions), or restructure.

3. Write pseudocode for ATM withdraw with insufficient funds check.

Model answer:

```
balance = 10000
read amount
IF amount <= 0:
  show "Invalid amount"
ELSE IF amount > balance:
  show "Insufficient funds"
ELSE:
  balance = balance - amount
  show "Withdrawn", amount, "New balance", balance
```

4. Explain while-else construct with PIN example.

Model answer: A `while` loop's `else` runs when the loop ends **normally** (condition becomes false) — **not** when you `break` out. PIN example: loop up to 3 attempts; on correct PIN `break`; if all attempts fail without break, `else` prints "Card blocked."

`python pin = "1234" attempts = 0 while attempts < 3: entered = input("PIN: ") attempts += 1 if entered == pin: print("Access granted") break else: print("Card blocked")`

Mini-Project (Assignment): HDFC Bank ATM Simulator (Console)

Title: Console ATM with balance + mini statement
Lab kit (build here): 15_Lab_Assets/08_atm_menu/

What to build

1. Starting balance ₹10,000 (float)
2. Menu: balance, deposit, withdraw, mini statement (last 3 transactions as strings), exit
3. Validate: no negative deposit/withdraw; no overdraft
4. Use `while True` + `break` on exit
5. Use list `transactions = []` to append strings like `"Deposited ₹500"`

Sample transaction log line:

```
[31-07-2026] Withdrawn ₹2000 | Balance ₹8000.00
```

Stretch

- Cap daily withdraw total at ₹20,000.
- Require PIN (`1234`) once before the menu loop.

Done when

- [ ] Lab kit ATM ticket passes (menu does not crash / exit wrongly)
- [ ] Deposit and withdraw update balance correctly with validation
- [ ] Mini statement shows up to last 3 transactions

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. Why is menu-driven design still used in training if real apps have GUIs?
2. Could you use `for` instead of `while` for a **HDFC Bank** ATM menu? What would be awkward?
3. How does nested if map to **Employee Management Portal** approval workflows (employee → manager → HR)?
4. What real-world system might run an infinite loop intentionally?
5. How would you add "confirm before **HDFC Bank** withdraw" using flow control only?

Q1. Explain if/elif/else execution order.

A: Python tests top to bottom; first true branch runs; rest skipped; else runs if none true.

Q2. Difference between break and continue?

A: break exits loop; continue skips to next iteration of same loop.

Q3. What is range(2, 10, 3)?

A: 2, 5, 8 — start 2, stop before 10, step 3.

Q4. How implement retry logic for login?

A: while attempts < max: prompt; if valid break; else increment attempts.

Q5. Nested loops use case?

A: Multiplication table (outer number, inner multiplier); grid rows/cols; batch processing per department.
